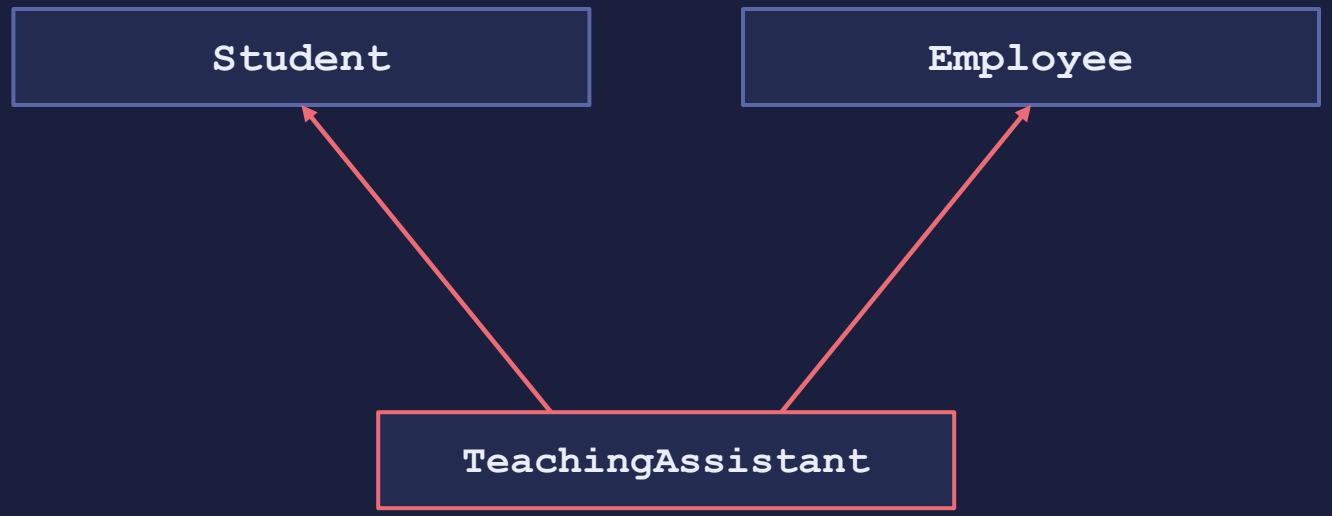
CIS 3300 · INHERITANCE AND POLYMORPHISM

Multiple Inheritance

Part 4 of 4: the diamond problem, and how Java gets most of the benefit through interfaces

More than one direct superclass



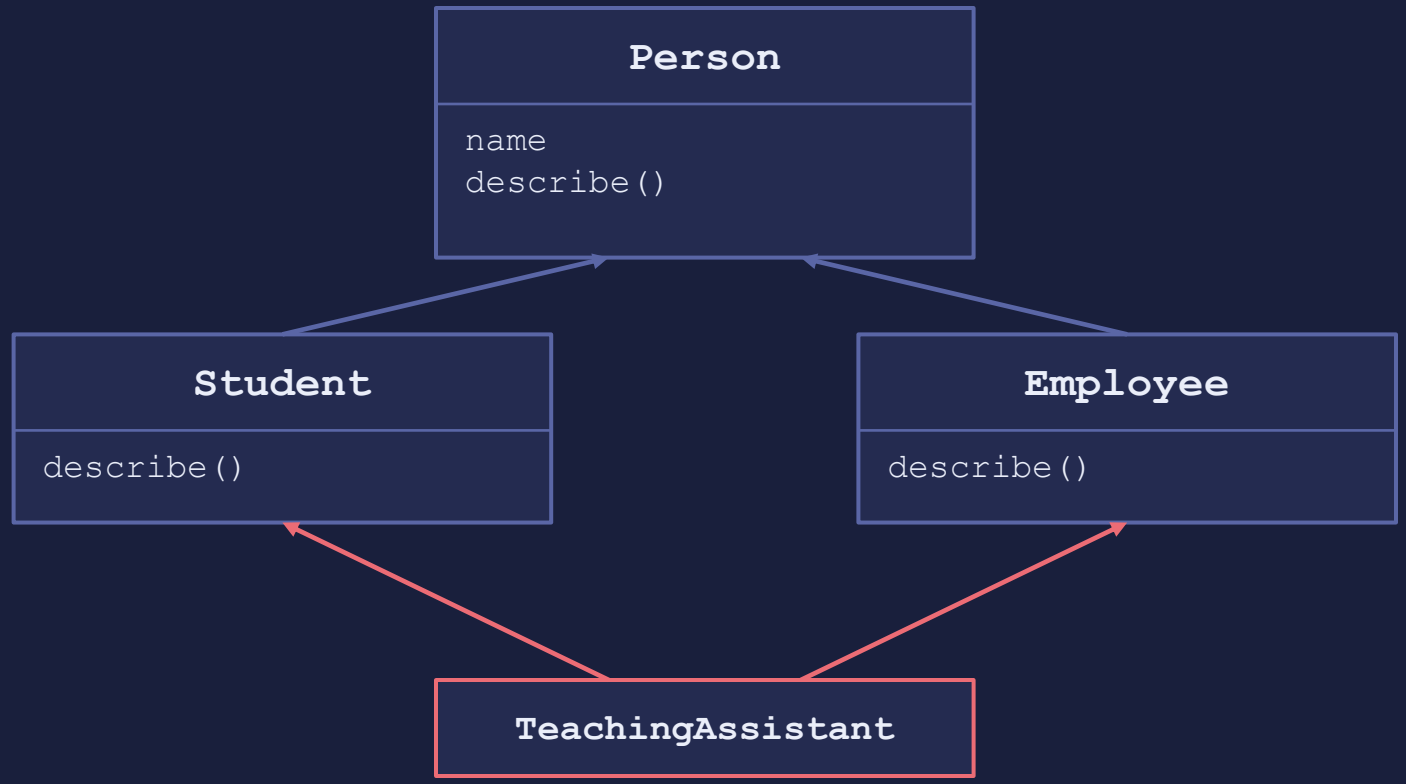
Why people want it

A teaching assistant really is both a student and an employee. C++ and Python allow this. The trouble starts when both parents share an ancestor.

```
class TeachingAssistant extends Student, Employee
```

Not legal Java

Two paths to one ancestor raise three questions



Behavior

Which describe() does it inherit?

State

One copy of name, or two?

Construction

Does Person() run once, or twice?

Split it three ways: state, behavior, and type

State

No

Only classes have instance fields, and a class extends one class.

Behavior

Limited

Since Java 8, default methods can come from several interfaces, with firm conflict rules.

Type

Yes

A class can implement any number of interfaces, and is an instance of each.

“Java does not support multiple inheritance” is true for state, partly true for behavior, and false for type.

THE RULES

A class extends one class and implements many interfaces

A class

- extends exactly one class (Object if none)
- implements any number of interfaces
- cannot extend an interface or implement a class

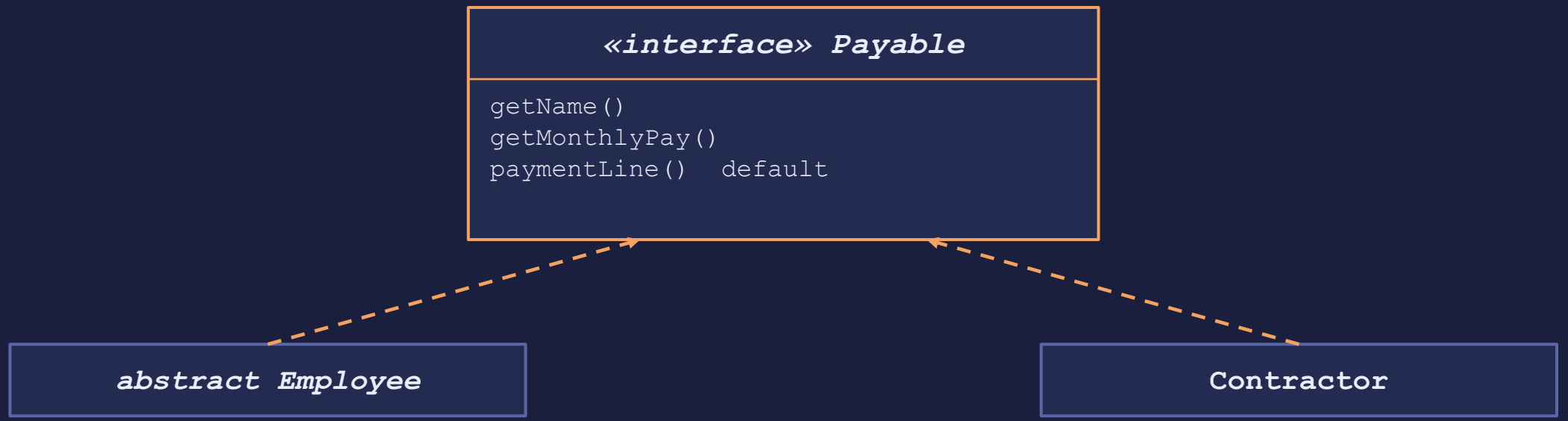
An interface

- extends any number of interfaces
- cannot extend a class
- cannot implement anything

```
public class Manager extends SalariedEmployee implements Mentor, Trainer
public interface Leader extends Mentor, Trainer
```

In a class header, extends comes first, then implements.

Payable gives employees and contractors one shared type

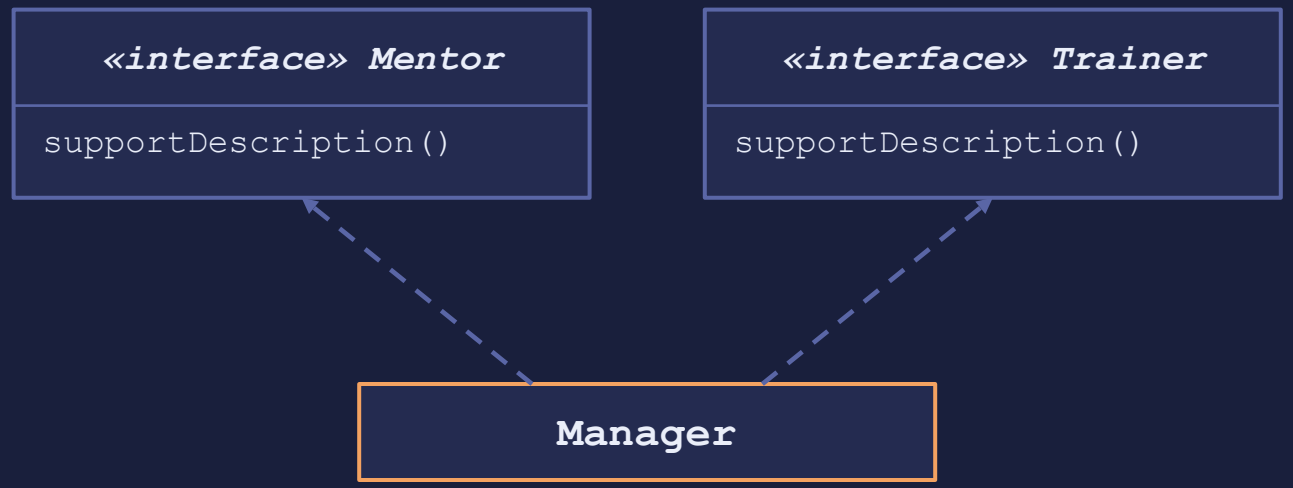


Salaried, Hourly, Manager, Executive: no changes

Not an Employee: no ID, no salary

```
ArrayList<Payable> payees = ...;   // Ada, Linus, Grace, Alan, Margaret
for (Payable p : payees) total += p.getMonthlyPay();   // 42950.00
```

Two default methods, one name: Java makes you choose




```
error: types Mentor and Trainer
  are incompatible; class Manager
  inherits unrelated defaults
```

```
@Override
public String supportDescription() {
    return Mentor.super.supportDescription() + " and "
         + Trainer.super.supportDescription();
}
```

The fix

Override the method. InterfaceName.super.method() calls one interface's default, so you can pick, combine, or replace.

CONFLICT RULES

Three rules settle every conflict

1. Classes win. A superclass method beats an interface default.
2. The more specific interface wins. A subinterface’s override beats its parent’s default.
3. Otherwise you must override, and you may call X.super.method().

Rule 1 is why an interface cannot have a default toString, equals, or hashCode: every class already inherits them from Object, so the default could never run.

Abstract class, interface, or has-a

Abstract class

Shared identity and shared data. One per class. Fields and constructors allowed.

Example: Employee

Interface

A capability or role. Any number per class. No instance fields, no constructors.

Example: Payable, Mentor, Trainer

Has-a

When the is-a sentence feels forced. Hold the other objects in fields instead.

Example: A TA has a Student record and an Employee record

NEXT

Next: build it in Example 5

1. Add Payable, and make Employee implement it.
2. Write Contractor, and pay everyone from one ArrayList<Payable>.
3. Add Mentor and Trainer to Manager; read the error; fix it.
4. Count the is-a tests an Executive passes.

Read the Part 4 notes and Core Java, Volume I, Sections 5.1 and 6.1.